



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Enterprise ICT Architectures **(Module 1) - First Project Delivery** **ER and SQL**

Author(s): **Alberto Lenoci 1 - 11095788**

Elena Casaleggi 2 - 10706547

Matteo Lo Giudice 3 - 10829241

Veronica Fatigati 4 - 11095787

Group Number: **4**

Academic Year: 2024-2025

Contents

Contents	i
1 Introduction	1
2 ER Model	3
2.1 <i>WellDressed Fashion Company ER Model</i>	3
3 Logical Model	7
3.1 <i>WellDressed Fashion Company LM</i>	9
4 Relational Model	13
4.1 <i>WellDressed Fashion Company RM</i>	13
5 DDL - Database Construction	23
6 Populating the Database	31
7 Queries	43
 List of Figures	 61
List of Tables	63

1 | Introduction

This project aims at creating a database for the *fashion company WellDressed*. The structure of the database is outlined by the following text:

“The WellDressed fashion company wants to create a database about its products and exhibitions. The company hires fashion models to partake in fashion exhibitions. Each model provides their personal data, including personal ID, name, surname, birth date, phone number, height, and dress size (XS, S, M, L, XL, etc.). Models are assigned to a fashion designer. These are described by their Personal ID, name, surname, stage name, phone number, and a brief description of their career. Fashion designers produce dresses for the models to wear in exhibitions. Each dress is worn by one model. Dresses are identified by a unique ID, size (XS, S, M, L, XL, etc.), colour, and production time. Dresses are made using fabrics described by a unique ID, price per square meter, name, and description of their properties. The amount of fabric used to produce a dress is also stored. Fabrics can be made from other fabrics. Models partake in exhibitions, described by a unique ID, duration (in minutes), title, location address, and description. Once a year, a fashion competition is held. The competition involves fashion designers submitting one of their dresses. The winning dress is marked as such. Dresses can be submitted to one competition only.”

To create a relational database, the process begins by designing an *ER Model*, followed by the definition of the *Logical Model*, and finally the *Relational Model*. Additionally, the SQL code used for the actual database construction in *MySQL Workbench* will be written and described based on the Relational Model. In order to provide a visual representation of the database, each table will be randomly populated as an example. Lastly, a series of SQL queries will be executed on the database to observe various behaviors and to assess the efficiency of the system.

2 | ER Model

An **Entity-Relationship (ER) Model** is a flowchart that shows the relationships between entities in a system, such as individuals, things, or ideas. ER diagrams illustrate the interdependence of entities, relationships, and their properties using a predetermined collection of symbols, including ovals, diamonds, rectangles, and connecting lines. Entities act as nouns and relationships act as verbs.

2.1. *WellDressed Fashion Company ER Model*

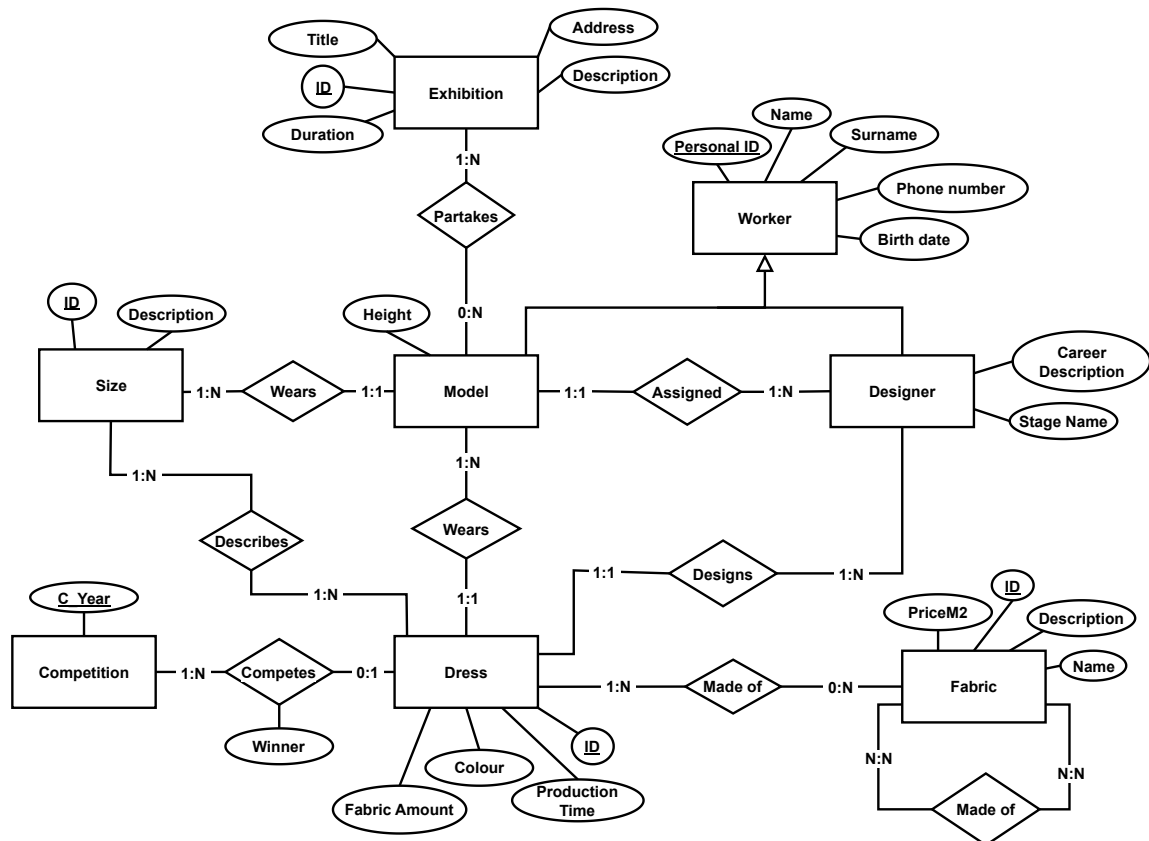


Figure 2.1: ER Model

Entity: Real-world object distinguishable from other objects. An entity is described using a set of Attributes.

Entity Set: A collection of entities of the same kind.

Relationship: Association among two or more entities.

Relationship Set: Collection of similar relationships.

Figure 2.1 represents the ER Model of *WellDressed fashion company* which we developed from the given text reported in the introduction and with the addition of several assumptions.

The first assumption we made regards the generalized entity "**Worker**". The relationship between workers and both models and designers is called an ISA hierarchy. Its properties include inheritance, as all attributes of the supertype apply to the subtype, as well as transitivity. Additionally, we assumed that the IsA relationship is total exclusive to ensure that a designer cannot be a model and vice versa. This decision simplifies the representation of shared attributes like name, surname, personal ID, and phone number without redundancy. For example, both models and designers are uniquely identified by a Personal ID. Although it wasn't stated in the original text, the designers' birthdate was added to ensure uniformity across worker types (since models have a birth date attribute). This makes it possible for all employees' personal information to be stored consistently.

Our second assumption was to represent "**Size**" as a separate entity with its own attributes like description (XS, S, M, L, XL) instead of making it an attribute of the dress. This allows for flexibility in the data model where multiple dresses can exist in various sizes, and multiple models can share the same size. The relationship between Size and Dress is in fact many-to-many (N), acknowledging that each dress can come in multiple sizes, and each size can correspond to several dresses. In addition, as stated in the given text, each dress is worn by only one model, but a model can wear multiple dresses (1:N relationship). This makes sense in the context of fashion exhibitions, where models showcase multiple outfits, but each dress is tailored to a specific model.

Another important reasoning to mention is the explanation behind the self-loop on fabrics or better the relationship between "**Fabric**" and itself (i.e., "fabrics can be made from other fabrics") is modeled as a many-to-many (N-N) relationship. This reflects real-world scenarios where one fabric might be composed of multiple others, and these constituent fabrics can be reused in various other compositions.

The third assumption we made explains why designers do not have a direct relationship with the competition entity since their involvement is tied to their dresses. This is be-

cause designers are responsible for creating dresses, which then compete in competitions, and therefore the **"Dress"** and **"Competition"** relationship is sufficient. Additionally, the requirement that each designer must have at least one dress ensures that designers contribute actively to the fashion showcases.

The **"winner"** attribute is logically placed on the **"competes"** relationship, as it is the participation of the dress in the competition that determines if it has won or not. This approach also avoids redundant attributes in the dress entity itself, since not every dress will participate in a competition, and only a small subset of dresses will ever be marked as winners. By attaching this attribute to the relationship instead of the dress entity, the model captures that a dress only wins relative to a specific competition, rather than implying a global status for the dress.

Lastly, we assumed from the given text that N number of models participate in exhibitions, meaning each model can partake in multiple exhibitions, and each exhibition can feature multiple models. This reflects the dynamic nature of fashion shows where models often participate in different events, and shows feature several models.

3 | Logical Model

This section presents the **Logical Model** of the project, preceded by a theoretical reflection. The Logical Data Model (LDM) defines the structure of information elements and their relationships, guiding the design of an information system. It bridges the gap between the *ER Model*, which outlines high-level requirements, and the *Relational Model*, which specifies how data will be stored and managed. This distinction ensures that the model remains independent of the underlying technology, promoting flexibility and adaptability to technological changes.

Unlike the conceptual model, the LDM provides more detailed information about the system's elements, specifying data and their interconnections. It accurately represents entities in a business domain, their attributes, and the relationships between entities, without delving into the technical implementation or data storage aspects.

The three fundamental components of the LDM are:

- **Entities:** Represent people, objects, or concepts relevant to the application domain. Each entity is defined to reflect business elements that the system must manage;
- **Relationships:** Describe the links between two or more entities, detailing the nature of associations and the rules governing such interactions;
- **Attributes:** Descriptive characteristics of entities that provide specific information to further define the entity and capture the necessary data for business processes.

Additionally, the model formalizes the concepts of **primary key (PK)** and **foreign key (FK)**, both of which are fundamental in defining and enforcing the integrity of relationships between entities. In particular:

- The **primary key** is a unique identifier for each record within an entity. It ensures that each instance of an entity is distinguishable from others, guaranteeing the uniqueness of every record and its correct identification within the database context.
- The **foreign key** is an attribute or set of attributes that creates a link between two entities by referencing the PK of another entity. This link establishes a relationship

between entities and ensures referential integrity of the data.

In the LDM, one of the key challenges is determining how to represent relationships between entities, particularly in scenarios where multiple types of relationships such as one-to-one (1:1), one-to-many (1:N) or many-to-one (N:1), and many-to-many (N:N) are involved. Each of these cases requires a tailored approach to ensure proper data management and integrity.

One-to-One (1:1) Relationship

In a one-to-one (1:1) relationship, an entity in one table is related to exactly one entity in another table, and vice versa. For this type of relationship, the FK can be placed in either of the two entities, as both are equally valid candidates. However, the decision on where to place the FK often depends on the specific business logic or practical considerations, such as minimizing data redundancy or improving query performance.

One-to-Many (1:N) or Many-to-One (N:1) Relationship

In a one-to-many (1:N) or many-to-one (N:1) relationship, the situation is more straightforward. The entity on the "many" side of the relationship will contain the FK that references the PK of the entity on the "one" side. This ensures that multiple records in the "many" entity can be associated with a single record in the "one" entity.

Many-to-Many (N:N) Relationship and the Bridge Table

A more complex case arises when dealing with many-to-many (N:N) relationships, where multiple instances of one entity can be associated with multiple instances of another. In this situation, it is not feasible to manage the relationship by simply adding a FK to either entity. Instead, a *Bridge Table* (or *Join Table*) is introduced to resolve this issue.

The *Bridge Table* acts as an intermediary that allows for the representation of many-to-many relationships between two entities. It does this by holding the PKs of the related entities as FKs within itself. This table serves as a link between the two entities, allowing the accurate representation of all possible associations between them.

The process for creating a *Bridge Table* typically involves the following steps:

1. **Identify the two entities involved:** Determine which two entities are participating in the (N:N) relationship.
2. **Create the bridge table:** Construct a new table that includes the PKs from both

entities as FKs.

3. **Define a composite Primary Key:** Combine the PKs of the two entities to form a composite PK in the Bridge Table, ensuring each combination is unique.

The *Bridge Table* is essential not only for maintaining the integrity of the data but also for providing a scalable solution for managing complex relationships. Additionally, it can store attributes specific to the relationship itself, making it a flexible and valuable structure within the model.

3.1. *WellDressed Fashion Company LM*

The following **Logical Model** has been developed for the *WellDressed fashion company*, with the objective of creating a database to manage data related to its products and exhibitions.

Model

Model ([Personal ID](#), Name, Surname, Phone Number, Birth Date, Height, [Size ID](#), [Designer ID](#))

The *Model* entity stores personal information about the fashion models participating in exhibitions. The PK of this entity is [Personal ID](#). It also includes two FKs: [Size ID](#) which references the *Size* entity, indicating the dress size associated with each model; and [Designer ID](#), which links to the *Designer* entity, identifying the designer connected to each model.

Designer

Designer ([Personal ID](#), Name, Surname, Phone Number, Birth Date, Career Description, Stage Name)

The *Designer* entity stores information about the fashion designers. The PK is [Personal ID](#).

Fabric

Fabric ([ID](#), Name, Description, PriceM2)

The *Fabric* entity stores information about fabric(s) used to produce dresses. The PK is [ID](#).

Fabric_Fabric

Fabric_Fabric ([Fabric ID1](#), [Fabric ID2](#))

The *Fabric_Fabric* entity is used to represent a relationship where one fabric is composed of, or derived from, another fabric. This structure is necessary to model situations where a fabric is not a standalone material but is instead made from combinations or blends of other fabrics.

The entity uses a composite PK formed by [Fabric ID1](#) and [Fabric ID2](#), which are also FKs that reference the *Fabric* entity. [Fabric ID1](#) represents the fabric being created, while [Fabric ID2](#) represents the component fabric(s) used in its production. A bridge table is necessary here to manage the potentially complex relationships where multiple fabrics can be combined to create a new fabric, and the same component fabric can be used in the creation of multiple other fabrics.

Exhibition

Exhibition ([ID](#), Title, Duration, Address, Description)

The *Exhibition* entity stores information about fashion exhibitions. The PK is [ID](#).

Model_Exhibition

Model_Exhibition ([Exhibition ID](#), [Model ID](#))

The *Model_Exhibition* entity represents the relationship between models and exhibitions. The composite PK is formed by [Exhibition ID](#) and [Model ID](#), both of which are also FKs that reference the *Exhibition* and *Model* entities, respectively.

Size

Size ([ID](#), Description)

The *Size* entity stores information about dress sizes. The PK is [ID](#).

Competition

Competition ([C Year](#))

The *Competition* entity stores information about the annual fashion competition. The PK is [C Year](#).

Competition_Dress

Competition_Dress ([Dress ID](#), [C Year](#), Winner)

The *Competition_Dress* entity represents the relationship between dresses and competitions. The composite PK is formed by [Dress ID](#) and [C Year](#), both of which are also FKs that reference the *Dress* and *Competition* entities. In the original ER model, the attribute Winner was part of the relationship between Dress and Competition, indicating which dress won a particular competition. In the logical model, this attribute has been retained as a field within the *Competition_Dress* bridge table. By doing so, it remains associated with the specific instance of a dress participating in a competition.

Dress

Dress ([ID](#), Production Time, Color, Fabric Amount, [Model ID](#), [Designer ID](#))

The *Dress* entity stores data related to dresses produced by fashion designers. The PK is [ID](#), while [Model ID](#) and [Designer ID](#) are FKs that reference the *Model* and *Designer* entities.

Dress_Size

Dress_Size ([Dress ID](#), [Size ID](#))

The *Dress_Size* entity represents the relationship between dresses and sizes. The composite PK is formed by [Dress ID](#) and [Size ID](#), both of which are also FKs that reference the *Dress* and *Size* entities.

Dress_Fabric

Dress_Fabric ([Dress ID](#), [Fabric ID](#))

The *Dress_Fabric* entity manages the fabrics used to produce each dress. The composite PK is formed by [Dress ID](#) and [Fabric ID](#), both of which are also FKs that reference the *Dress* and *Fabric* entities.

4 | Relational Model

This fourth section of the project is dedicated to the Relational Model, developed based on the Logical Model, which was thoroughly examined in the preceding section. The transition from the Logical Model to the Relational Model represents a crucial step in the database design process. Specifically, the elements defined within the Logical Model are systematically transformed into their relational counterparts through a structured procedure. In this process, entities are converted into tables, while the attributes of these entities become the columns of the corresponding tables. The Primary Keys identified in the Logical Model are implemented as columns in the Relational Model, ensuring the unique identification of each row within the database. An equally fundamental aspect of this transition is the definition of Foreign Keys. These attributes play a pivotal role in establishing relationships between tables, guaranteeing the relational integrity of the database.

4.1. *WellDressed Fashion Company RM*

Here a subset of the database tables of the *WellDressed* Fashion Company is presented, comprising a limited number of entries to maintain clarity and readability. Each table is labeled with a title corresponding to the entity or relation it represents. Primary keys are written in blue and underlined, while foreign keys are marked in orange and underlined.

Model

<u>Personal ID</u>	Name	Surname	Phone Number
1	Erick	Tapp	674-417-1273
2	Sibby	Essery	307-736-8666
3	Sile	McCritichie	855-464-3139

Table 4.1: Part 1: Table "Model"

Birth Date	Height (m)	<u>Size ID</u>	<u>Designer ID</u>
10/10/1991	1.83	1	22
8/27/1991	1.75	3	11
2/20/1996	1.83	4	7

Table 4.2: Part 2: Table "Model"

The **Model** table contains detailed information about fashion models, including personal data, physical attributes, and their relationships with other entities in the database. The table is structured as follows:

- **Personal ID**: A unique identifier assigned to each model, ensuring that each entry in the table represents a distinct individual.
- **Name**: The first name of the model.
- **Surname**: The last name or family name of the model.
- **Phone Number**: The contact phone number of the model, which may include various formats based on the country or personal preference.
- **Birth Date**: The date of birth of the model, which can be used to derive their age or for other temporal analyses.
- **Height (m)**: The height of the model, recorded in meters.
- **Size ID**: This field represents a reference to the corresponding size information for the model. It links to another table that stores the details about various clothing or accessory sizes.
- **Designer ID**: This field establishes a relationship between the model and a specific fashion designer, referring to another table where the details of designers are stored. It indicates which designer the model has worked with.

Designer

<u>Personal ID</u>	Name	Surname	Phone Number	Birth Date
1	Terra	Akast	623-967-8617	10/17/1982
2	Reeva	Gullane	711-860-4324	11/8/1963
3	Myrlene	MacDermid	445-733-6647	5/10/2000

Table 4.3: Part 1: Table "Designer"

Career Description	Stage Name
Dynamic individual with strong leadership skills	Gio
Dynamic individual with strong leadership skills	Eva
Experienced professional with a proven track record	Luna

Table 4.4: Part 2: Table "Designer"

The **Designer** table stores comprehensive information concerning dress designers, comprising their personal details such as Name, Surname, Birth Date and Phone Number, together with a brief description of their career. Their stage name is also included in the table, which is structured as follows:

- **Personal ID**: As mentioned above for the *Model* entity, this is a unique identifier assigned to every designer, indicating distinct individuals.
- **Name**: The First Name of the designer.
- **Surname**: The Family name of the designer.
- **Phone Number**: The contact phone number of the designer, which may include various formats based on the country or personal reference.
- **Birth Date**: The date of birth of the designer, useful to derive age and other temporal analyses.
- **Career Description**: This is a brief description of the specialist figure of the individual designer, including some relevant skills in the field that may be employed to derive professional considerations.
- **Stage Name**: This is the Stage Name picked by every designer and it may be employed to easily connect them to their artistic creations.

Fabric

<u>ID</u>	Name	Description	PriceM2 (m ²)
1	Cotton	Soft	62.01
2	Silk	Luxurious	178.88
3	Linen	Breathable	55.73

Table 4.5: Tabel "Fabric"

The **Fabric** table stores detailed information on the entity, displaying the unique ID number of each fabric, corresponding to their name and accompanied by a brief description and a price indicator.

- **ID**: This is the code for unique identification of each fabric in the table.
- **Name**: This indicates the denominations of the fabrics.
- **Description**: One-adjective description of the main characteristic of each fabric in the table.
- **Price m^2** : This gives indication in € on the monetary price per square metre of each of the fabrics listed in the table.

Fabric_Fabric

<u>Fabric ID1</u>	<u>Fabric ID2</u>
14	1
9	13
10	11

Table 4.6: Table "Fabric_Fabric"

The bridge table **Fabric_Fabric** consists in two columns, representing the relation between different fabrics, and is structured as follows:

- **Fabric ID1**: This attribute indicates the ID of the fabrics listed in the table.
- **Fabric ID2**: This attribute too gives information regarding the ID of the fabrics listed in the table.

This table represents the entity **Exhibition**, including information about the ID number and Title, and some time and locational details about each events, developed as follows:

Exhibition

<u>ID</u>	Title	Duration	Address	Description
1	Prism of Style	120	928 Esker Parkway	Practice Scrimmage
2	Radiant Allure	60	00691 Prentice Plaza	Pre-season Friendly
3	Bright Allure	45	52 Becker Road	Practice Scrimmage

Table 4.7: Table "Exhibition"

- **ID**: This attribute represents the unique ID code for each exhibition.
- **Title**: Title here refers to the denomination chosen for each different exhibition.
- **Duration**: This feature indicates the duration in minutes of every exhibition, in order to allow for temporal evaluations.
- **Address**: The address indicates the place where each exhibition included in the table is held, it gives indications to guests on where to go.
- **Description**: This column gives a brief description of each show.

Model_Exhibition

<u>Exhibition ID</u>	<u>Model ID</u>
1	8
2	10
3	18

Table 4.8: Table "Model_Exhibition"

The table **Model_Exhibition** represents the relation between each event entity and the corresponding model entity participating to it. It is structured as follows:

- **Exhibition ID**: This represents the unique ID code of the exhibition and permits to univocally refer to it.
- **Model ID**: This is the Personal ID of the models, and allows to uniquely identify the model and relate it to the ID of the exhibition.

Size

<u>ID</u>	Description
1	X-Small
2	Small
3	Medium
4	Large
5	X-Large

- **ID**: In this case, this is the unique ID for each of the sizes included in the table.
- **Description**: This column provides the standard clothing sizes, from X-Small to X-Large.

Competition

<u>C Year</u>
2020
2021
2022

Table 4.10: Table "Competition"

The **Competition** table represents the entity *Competition*, uniquely identified by the year in which each one has been held.

- **C Year**: This is a unique indication to each competition, since they are held once per year, the reference is performed through the year.

Competition_Dress

<u>Dress ID</u>	<u>C Year</u>	Winner
5	2020	FALSE
17	2021	TRUE
23	2022	TRUE

Table 4.11: Table "Competition_Dress"

The **Competition_Dress** table represents the relation between competitions and dresses, including information that enable to uniquely identify dresses, competitions and whether they won or not, structured as follows:

- **Dress ID**: This attribute is the unique ID of the dresses created by the designers.
- **C Year**: This attribute refers to the year in which every competition has been held.
- **Winner**: This is a Boolean attribute and can only be TRUE or FALSE, indicating whether a certain dress has won or not the competition of that year.

Dress

<u>ID</u>	Production Time	Color	Fabric Amount (m ²)	<u>Model ID</u>	<u>Designer ID</u>
1	1	Red	5	1	1
2	4	Turquoise	5	2	2
3	3	Goldenrod	1	3	3

Table 4.12: Table "Dress"

The **Dress** table provides detailed information about each dress, including their ID number and some information about production time and fabric amount, including the color and a reference to the IDs of corresponding models and designers.

- **ID**: Unique identification number for each dress.
- **Production Time**: This time is measured in days and gives indication about the time it took to produce each dress.
- **Color**: This column gives information regarding the color of each dress.
- **Fabric Amount (m²)**: This is a numeric indication in squared metres on the amount of fabric required to produce each dress included in the table.
- **Model ID**: this field establishes a relationship between the dress and a specific model through their personal IDs, referring to the entity *Model*, where these details are stored.
- **Designer ID**: This value is the Personal ID number of designers, representing a connection between the entity and the personal ID of each designer, referring to the table for the entity *Designer*, where this information is stored.

Dress _ Size

<u>Dress ID</u>	<u>Size ID</u>
1	3
2	5
3	3

Table 4.13: Table "Dress _ Size"

The **Dress _ Size** table represents the relation between the two entities *Dress* and *Size*, deployed through their unique IDs:

- **Dress ID**: This attribute is the unique ID of each individual dress.
- **Size ID**: This attribute is the unique ID of each one of the 5 different sizes.

Dress _ Fabric

<u>Dress ID</u>	<u>Fabric ID</u>
1	25
2	1
3	22

Table 4.14: Table "Dress _ Fabric"

The **Dress _ Fabric** represents the relation between the two entities *Dress* and *Fabric*, displayed through their corresponding IDs as follows:

- **Dress ID**: As already mentioned above, this attribute refers to the unique ID of the entity Dress.
- **Fabric ID**: This attribute refers to the unique ID of the entity Fabric.

To further represent the Relational Model we obtained Figure 4.1 below through MySQL Workbench using the *Reverse Engineering* function, following the construction of the database described in the next chapter.

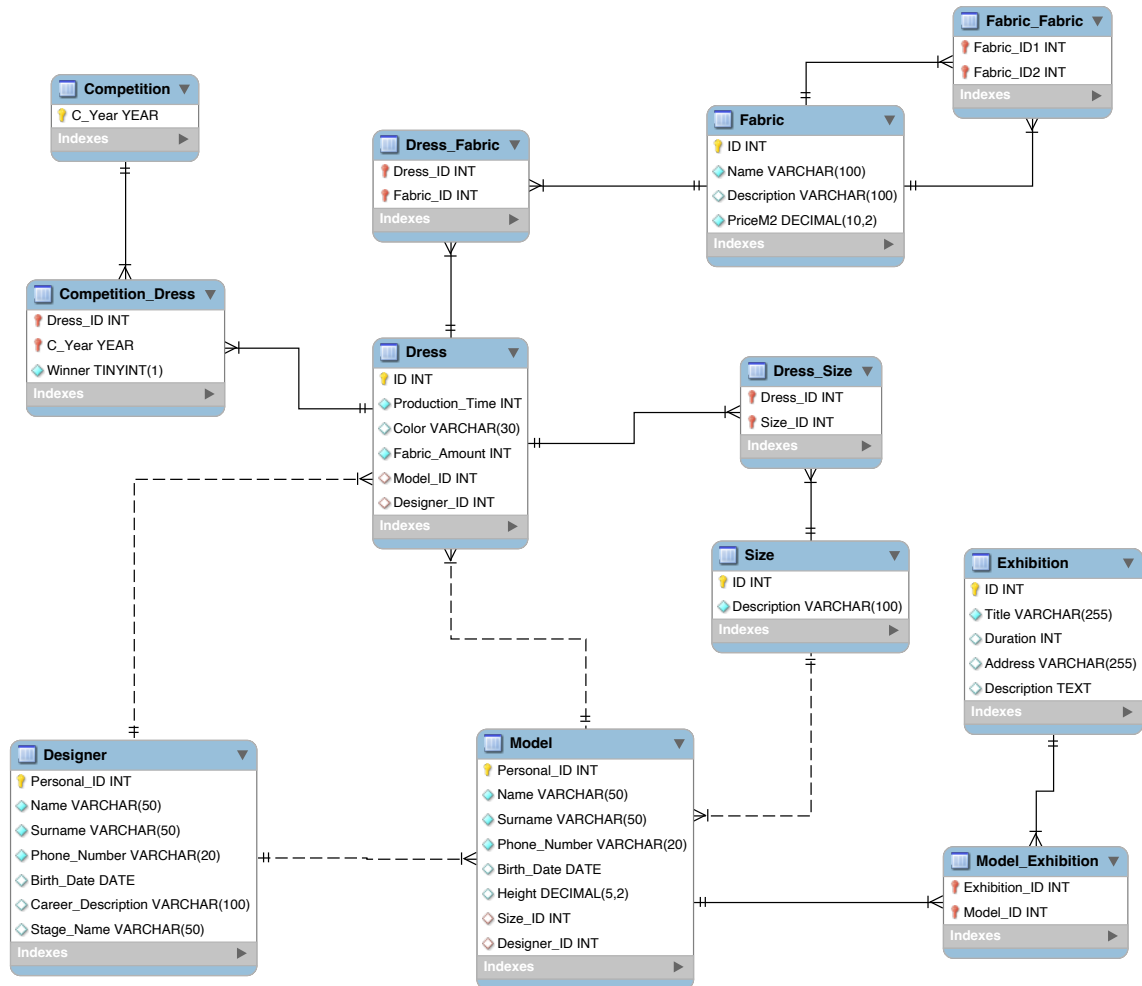


Figure 4.1: Relational Model by MySQL Workbench

5 | DDL - Database Construction

Preliminary Steps

The next step in creating a functioning database is the actual construction of the database, as required in the fourth request in the project assignment. As the scope of this project is to ultimately query the database with SQL, the entire process will be executed using the MySQL Workbench software and its (local) server database storage capabilities.

The three models we have previously defined allow us to easily identify the steps we need to take to properly store our data in tables and guarantee a correct behavior of the database when faced with queries of any type, both simple and complex.

The logical model, in particular, helps us define the "bridge tables" that will give the database the capabilities to handle the many "N to N" relationships present in the ER Model. These tables will be analyzed in further detail as they become relevant in the later parts of this chapter.

The database construction begins with the creation and selection of our "Schema", the canvas upon which our tables and relationships will be placed. The SQL code required for this operation is as follows:

```
1 CREATE DATABASE Project1;  
2 USE Project1;
```

We can now begin to create our tables. Each table will be modeled following the relational model, being named after each entity and assigning the different attributes to its columns. We will have to pay close attention to the data type of the different columns. Choosing the correct data types will guarantee compatibility with our dataset allow us to perform more detailed queries. We'll analyze the data types case by case.

5.1. Designer Table

Our first table will be the "Designer" table. As already underlined in the logical and relational model chapters, the total and exclusive hierarchy present in the ER model

allows us to completely separate the Model and Designer tables without the need of bridge tables.

```
1 CREATE TABLE Designer (  
2     Personal_ID INT AUTO_INCREMENT,  
3     Name VARCHAR(50) NOT NULL,  
4     Surname VARCHAR(50) NOT NULL,  
5     Phone_Number VARCHAR(20) NOT NULL UNIQUE,  
6     Birth_Date DATE,  
7     Career_Description VARCHAR(100),  
8     Stage_Name VARCHAR(50),  
9     PRIMARY KEY (Personal_ID)  
10 );
```

Our first operation is creating a column for each attribute assigned to this entity, following the models we have already created. We then select the [Primary Key](#) of this table: [Personal ID](#).

The data types used are as follows:

- **VARCHAR(n):** Indicates a string with a maximum length limit. The character limits chosen should cover all possible use cases of the Name, Surname, Stage Name, Phone Number, and Career Description fields.
- **DATE:** Indicates a date value. With the correct format in the dataset, this allows for the use of logical operators in subsequent queries.
- **INT:** Used for the Personal ID, as this value is represented by a simple number. It allows for numerical computation in queries.

We have also included a number of different constraints in order to preserve the integrity of the database should a future expansion of the data set occur. These are:

- **NOT NULL:** The field cannot have a null value. This prevents the creation of unidentifiable entities within our database.
- **UNIQUE:** The field cannot have the same value as another. As this is used only for the phone number field, this assumes that every individual entity must have its own phone number and prevents cases of possible abuse. This is not used for the Personal ID field, as it is the [Primary Key](#) of this table and thus automatically unique.
- **AUTO INCREMENT:** This constraint is applied to the [Primary Key](#) in order to prevent database malfunctions in the event of inaccurate inputs and maintain

integrity.

5.2. Size Table

We repeat the same process for the "Size" table.

```
1 CREATE TABLE Size (  
2     ID INT AUTO_INCREMENT,  
3     Description VARCHAR(100) NOT NULL,  
4     PRIMARY KEY (ID)  
5 );
```

The [Primary Key](#) is [ID](#). The reasoning behind the constraints and the data types are the same as above.

5.3. Model Table

After creating these two tables, we can now move on to the "Model" table. This table, as seen in the logical model, presents our first [foreign keys](#). These keys are connected to the "Designer" and "Size" tables. Had we not created them beforehand, the SQL code would have returned an error and fail its operation, requiring us to otherwise extend our table creation process with more steps.

The code for the creation of the "Model" table is as follows:

```
1 CREATE TABLE Model (  
2     Personal_ID INT AUTO_INCREMENT,  
3     Name VARCHAR(50) NOT NULL,  
4     Surname VARCHAR(50) NOT NULL,  
5     Phone_Number VARCHAR(20) NOT NULL UNIQUE,  
6     Birth_Date DATE,  
7     Height DECIMAL(5,2),  
8     Size_ID INT,  
9     Designer_ID INT,  
10    PRIMARY KEY (Personal_ID),  
11    FOREIGN KEY (Size_ID) REFERENCES Size(ID) ON DELETE SET NULL,  
12    FOREIGN KEY (Designer_ID) REFERENCES Designer(Personal_ID) ON DELETE  
    SET NULL  
13 );
```

The [Primary Key](#) is [Personal ID](#). As this is under the ISA hierarchy of the "Designer" table, it shares many of the same attributes, but there are also some new ones with new data types.

- **DECIMAL (n,m):** This data type is used for the Height attribute. It allows the

numerical data to be stored as a floating point and be correctly computed inside more complex queries.

The [foreign keys](#) are [Size ID](#) and [Designer ID](#) and are created by referencing the respective [primary keys](#) in the respective tables. We have also added special constraints to them:

```
1 ON DELETE SET NULL
```

These constraints allow for concatenation of actions between tables. Should a [Designer ID](#) be deleted in the "Designer" table, the corresponding [foreign key](#) in the "Model" table will also be deleted, leaving the field with a null value. This keeps the integrity of the database stable in the case of a modification of the data set, preventing "empty" relationships to erroneously persist.

5.4. Fabric Table

```
1 CREATE TABLE Fabric (
2     ID INT AUTO_INCREMENT,
3     Name VARCHAR(100) NOT NULL,
4     Description VARCHAR(100),
5     PriceM2 DECIMAL(10,2) NOT NULL,
6     PRIMARY KEY (ID)
7 );
```

The [Primary Key](#) is [ID](#). Data types and constraints follow the same reasonings explained in the previous paragraphs.

5.5. Fabric_Fabric Table

This the first "bridge" table of our database, used to correctly represent the "N to N" relationships of the Fabrics themselves. It is also a special bridge table, as it corresponds to a self referencing relationship, but this will not require any special coding.

```
1 CREATE TABLE Fabric_Fabric (
2     Fabric_ID1 INT,
3     Fabric_ID2 INT,
4     PRIMARY KEY (Fabric_ID1, Fabric_ID2),
5     FOREIGN KEY (Fabric_ID1) REFERENCES Fabric(ID) ON DELETE CASCADE,
6     FOREIGN KEY (Fabric_ID2) REFERENCES Fabric(ID) ON DELETE CASCADE
7 );
```

The [Primary Key](#) is [Fabric ID1 ' Fabric ID2](#). The [foreign keys](#) are [Fabric ID1](#) and [Fabric ID2](#). This is what makes bridge tables special compared to other tables: The combination of their [foreign keys](#) is their [Primary Key](#).

(The primary key has been written with a quotation mark in order to not disrupt the underlining. This is very important to correctly indicate that the primary key is a single composite one, instead of two different and separated ones.)

Since this table represents a self referencing relationship, the foreign keys are the same primary keys from the "Fabric" table, but are stored under two different aliases.

The foreign keys constraints also present a slight variation from our previous case:

```
1 ON DELETE CASCADE
```

The "CASCADE" command signals the database to delete the entire table row in case a Fabric ID is deleted from the main "Fabric" table.

Data types and constraints follow the same reasonings explained in the previous paragraphs.

5.6. Exhibition Table

```
1 CREATE TABLE Exhibition (
2     ID INT AUTO_INCREMENT,
3     Title VARCHAR(255) NOT NULL,
4     Duration INT,
5     Address VARCHAR(255),
6     Description TEXT,
7     PRIMARY KEY (ID)
8 );
```

The Primary Key is ID. Data types and constraints follow the same reasonings explained in the previous paragraphs.

5.7. Model_Exhibition Table

The second bridge table of our database.

```
1 CREATE TABLE Model_Exhibition (
2     Exhibition_ID INT,
3     Model_ID INT,
4     PRIMARY KEY (Exhibition_ID, Model_ID),
5     FOREIGN KEY (Exhibition_ID) REFERENCES Exhibition(ID) ON DELETE
6     CASCADE,
7     FOREIGN KEY (Model_ID) REFERENCES Model(Personal_ID) ON DELETE
8     CASCADE
9 );
```

The Primary Key is Exhibition ID ' Model ID. The foreign keys are Exhibition ID and Model ID. This follows the same reasoning for primary keys and foreign keys explained in the previous paragraphs.

Data types and constraints follow the same reasonings explained in the previous paragraphs.

5.8. Competition Table

```
1 CREATE TABLE Competition (
2   C_Year YEAR PRIMARY KEY
3 );
```

The Primary Key is C_Year. Its data type is:

- **YEAR:** Functions similar to the **DATE** data type, but only supports year formats. Allows for date computation in queries.

5.9. Dress Table

```
1 CREATE TABLE Dress (
2   ID INT AUTO_INCREMENT,
3   Production_Time INT NOT NULL,
4   Color VARCHAR(30),
5   Fabric_Amount INT NOT NULL,
6   Model_ID INT,
7   Designer_ID INT,
8   PRIMARY KEY (ID),
9   FOREIGN KEY (Model_ID) REFERENCES Model(Personal_ID) ON DELETE SET
10  NULL,
11  FOREIGN KEY (Designer_ID) REFERENCES Designer(Personal_ID) ON DELETE
12  SET NULL
13 );
```

The Primary Key is ID. The foreign keys are Model ID and Designer ID. This follows the same reasoning for primary keys and foreign keys explained in the previous paragraphs.

Data types and constraints follow the same reasonings explained in the previous paragraphs.

5.10. Competition_Dress Table

Third Bridge table of our dataset. As explained in the relational model chapter, this does not represent a "N to N" relationship, but its existence is still required due to "Winner"

being and attribute of the relationship between the "Dress" and "Competition" tables.

```

1 CREATE TABLE Competition_Dress (
2     Dress_ID INT,
3     C_Year YEAR,
4     Winner BOOLEAN NOT NULL,
5     PRIMARY KEY (Dress_ID, C_Year),
6     FOREIGN KEY (Dress_ID) REFERENCES Dress(ID) ON DELETE CASCADE,
7     FOREIGN KEY (C_Year) REFERENCES Competition(C_Year) ON DELETE
  CASCADE
8 );

```

The Primary Key is Dress ID ' C_Year. The foreign keys are Dress ID and C_Year. This follows the same reasoning for primary keys and foreign keys explained in the previous paragraphs.

The "Winner" attribute presents a new data type:

- **BOOLEAN:** A Boolean value only allows two data values: "TRUE" and "FALSE". Inside SQL, Boolean is not directly supported, but it is converted into the equivalent data type **SMALLINT**, in which TRUE=1 and FALSE=0. The boolean data type allows us to easily check if a Dress won a certain competition or not

Other data types and constraints follow the same reasonings explained in the previous paragraphs.

5.11. Dress_Size Table

Fourth bridge table of our database.

```

1 CREATE TABLE Dress_Size (
2     Dress_ID INT,
3     Size_ID INT,
4     PRIMARY KEY (Dress_ID, Size_ID),
5     FOREIGN KEY (Dress_ID) REFERENCES Dress(ID) ON DELETE CASCADE,
6     FOREIGN KEY (Size_ID) REFERENCES Size(ID) ON DELETE CASCADE
7 );

```

The Primary Key is Dress ID ' Size ID. The foreign keys are Dress ID and Size ID. This follows the same reasoning for primary keys and foreign keys explained in the previous paragraphs.

Data types and constraints follow the same reasonings explained in the previous paragraphs.

5.12. Dress_Fabric Table

Fifth bridge table and final table of our database.

```
1 CREATE TABLE Dress_Fabric (  
2     Dress_ID INT,  
3     Fabric_ID INT,  
4     PRIMARY KEY (Dress_ID, Fabric_ID),  
5     FOREIGN KEY (Dress_ID) REFERENCES Dress(ID) ON DELETE CASCADE,  
6     FOREIGN KEY (Fabric_ID) REFERENCES Fabric(ID) ON DELETE CASCADE  
7 );
```

The Primary Key is Dress ID ' Fabric ID. The foreign keys are Dress ID and Fabric ID. This follows the same reasoning for primary keys and foreing keys explained in the previous paragraphs.

Data types and constraints follow the same reasonings explained in the previous paragraphs.

6 | Populating the Database

This section illustrates how the database was populated using Mockaroo. For each table representing entities and bridge tables, we generated 25 tuples, with the exception of the Size table, which was populated with only five options, as generating 25 examples in this case would not be meaningful.

Attached below are the screenshots of all the tables, which include all the values generated, unlike the previous chapter where only a few examples were provided for each.

ID	Description
1	X-Small
2	Small
3	Medium
4	Large
5	X-Large

Figure 6.1: Size's Table

Personal_ID	Name	Surname	Birth_Date	Phone_Number	Career_Description	Stage_Name
1	Terra	Akast	1982-10-17	623-967-8617	Dynamic individual with strong leadership skills	Gio
2	Reeva	Gullane	1963-11-08	711-860-4324	Dynamic individual with strong leadership skills	Eva
3	Myrlene	MacDermid	2000-05-10	445-733-6647	Experienced professional with a proven track record	Luna
4	Blondy	Evreux	1981-04-06	590-704-0130	Innovative thinker with a passion for problem-solving	Axel
5	Pegeen	Adcock	1996-10-02	724-692-8885	Dynamic individual with strong leadership skills	Aldo
6	Tull	Crosland	1999-11-05	806-718-8588	Innovative thinker with a passion for problem-solving	Star
7	Rancell	Antowski	1997-07-15	445-508-6525	Experienced professional with a proven track record	Echo
8	Ludovico	Knibley	1991-11-08	791-278-9944	Innovative thinker with a passion for problem-solving	Shadow
9	Tally	Darwent	1972-10-16	903-641-7473	Innovative thinker with a passion for problem-solving	Stella
10	Ingamar	Sabban	1997-11-23	328-674-2203	Innovative thinker with a passion for problem-solving	Agata
11	Leshia	Davidson	1996-08-31	161-559-0874	Innovative thinker with a passion for problem-solving	Amy
12	Jamil	Finnigan	1986-12-09	494-759-8733	Dynamic individual with strong leadership skills	Lola
13	Delila	Narramore	1998-05-20	412-634-7577	Experienced professional with a proven track record	Sonia
14	Mort	Rubinivitz	1963-01-22	214-908-4782	Dynamic individual with strong leadership skills	Luis
15	Jasen	McLeoid	1967-09-24	733-943-6689	Dynamic individual with strong leadership skills	Emily
16	Dulcinea	Brabant	1979-05-05	516-933-0284	Innovative thinker with a passion for problem-solving	Wilson
17	Marianna	Ziemens	1972-01-12	580-208-9255	Innovative thinker with a passion for problem-solving	Bobby
18	Waylen	Macer	1972-01-19	556-970-1186	Experienced professional with a proven track record	Sparkle
19	Malvina	Phil	1967-09-07	386-102-0520	Experienced professional with a proven track record	Rocky
20	Jennie	Kittow	1978-06-23	603-481-8027	Dynamic individual with strong leadership skills	Iris
21	Cherrita	Sharpin	1960-03-21	141-559-4082	Dynamic individual with strong leadership skills	Chris
22	Loralie	Alloway	1987-01-26	472-845-4812	Experienced professional with a proven track record	Andy
23	Danna	Metheringham	1970-11-05	415-268-6789	Dynamic individual with strong leadership skills	Heidi
24	Andee	Starrs	1966-01-31	405-748-6907	Experienced professional with a proven track record	Blaze
25	Goldina	Palffy	1999-02-26	356-394-3339	Dynamic individual with strong leadership skills	Wanda

Figure 6.2: Designer's Table

ID	Name	Description	Price M2 (€)
1	Cotton	Soft	62,01
2	Silk	Luxurious	179,28
3	Linen	Breathable	56,13
4	Wool	Lightweight	98,12
5	Polyester	Durable	47,15
6	Satin	Shiny	187,13
7	Velvet	Textured	112,51
8	Denim	Smooth	144,06
9	Chiffon	Stretchy	199,22
10	Jersey	Rich	126,01
11	Organza	Sheer	130,35
12	Tulle	Warm	11,18
13	Twill	Elegant	110,35
14	Cashmere	Cozy	181,17
15	Brocade	Versatile	9,3
16	Rayon	Crisp	40,22
17	Crepe	Flowing	17,31
18	Lycra	Vibrant	172,32
19	Nylon	Classic	20,21
20	Georgette	Sleek	15,37
21	Canvas	Comfy	58,59
22	Suede	Fluid	50,36
23	Fleece	Resilient	7,27
24	Gabardine	Plush	63,09
25	Corduroy	Structured	192,17

Figure 6.3: Fabric's Table

Personal_ID	Name	Surname	Phone_Number	Birth_Date	Height	Size_ID	Designer_ID
1	Erick	Tapp	674-417-1273	1991-10-10	183.05	1	22
2	Sibby	Essery	307-736-8666	1991-08-27	175.22	3	11
3	Sile	McCritichie	855-464-3139	1996-02-20	183.30	4	7
4	Ginny	Shuttell	510-208-3055	1991-04-20	170.04	2	21
5	Alfons	Tetford	908-681-2200	2000-10-08	178.22	5	3
6	Terry	Cohrs	637-137-6030	2001-12-14	169.27	1	12
7	Leyla	Foakes	331-180-1656	1990-09-19	163.60	3	19
8	Maryann	Botler	510-667-4481	1991-09-07	170.36	2	25
9	Delmor	Marchent	605-863-0885	1997-05-04	165.34	5	14
10	Goldia	Colnett	444-895-1500	1990-03-24	175.50	4	2
11	Ignacius	Goatman	519-524-9986	1996-08-08	176.28	3	16
12	Arvie	Leverett	260-715-1335	2000-08-14	181.23	2	18
13	Marja	Flaubert	207-106-5997	2003-11-01	181.45	5	17
14	Frayda	Cristofalo	917-870-0358	2004-04-19	178.33	1	20
15	Waylin	Mackilpatrick	341-311-9379	1994-01-16	179.11	4	15
16	Vanny	Tweedell	591-934-6299	1990-07-20	183.09	3	9
17	Rogers	Cowsby	222-832-0692	2000-03-02	169.21	4	24
18	Adorne	Couche	280-307-1294	2001-01-28	166.09	2	10
19	Dorthy	Castleman	535-802-7256	1994-10-04	160.20	5	1
20	Rebekkah	Neenan	529-601-0594	1995-03-03	165.12	1	23
21	Clarke	Alexsandrev	899-963-7164	1999-03-21	176.29	1	4
22	Cris	Buttwell	244-542-3914	1997-09-30	177.08	4	6
23	Goldie	Hourican	481-441-1145	1997-12-05	163.51	2	5
24	Orv	Delamaine	569-787-9832	1990-02-28	164.55	3	8
25	Gasparo	Thorouggood	282-566-8082	1992-10-08	187.39	5	13

Figure 6.4: Model's Table

Fabric ID1	Fabric ID2
14	1
9	13
10	11
24	19
8	2
24	17
18	24
1	14
9	4
25	22
11	5
10	18
3	17
6	15
6	18
8	11
4	10
14	8
19	17
22	12
10	10
13	25
14	9
25	10
18	23

Figure 6.5: Fabric_Fabric's Table

ID	Title	Duration	Address	Description
1	Prism of Style	120	928 Esker Parkway	Practice Scrimmage
2	Radiant Allure	60	00691 Prentice Plaza	Pre-season Friendly
3	Bright Allure	45	52 Becker Road	Practice Scrimmage
4	Midnight Glamour	30	1980 Hoard Alley	Practice Scrimmage
5	Rebel Chic	180	906 Monument Trail	Practice Scrimmage
6	Naive Chic	45	02 Longview Road	Spring Training Game
7	Golden Hour	30	04 Starling Center	Pre-season Friendly
8	Fluid Motion	30	2 Russell Avenue	Spring Training Game
9	Rebel Chic	30	1 Randy Road	Practice Scrimmage
10	Mystic Edge	120	4 Nelson Place	Pre-season Friendly
11	Rebel Chic	180	99 Main Street	Spring Training Game
12	Luxe Nomad	180	7925 Golf Plaza	Pre-season Friendly
13	Threads of Fantasy	60	66630 Randy Plaza	Practice Scrimmage
14	Golden Hour	30	3 Eastwood Hill	Spring Training Game
15	Joy Bloom	60	56613 Londonderry Parkway	Spring Training Game
16	Desert Bloom	45	237 Petterle Road	Practice Scrimmage
17	Echoes of Couture	60	1 Carpenter Terrace	Spring Training Game
18	Metropolis Muse	45	24 Cordelia Hill	Pre-season Friendly
19	Future Vogue	30	09 Hudson Avenue	Pre-season Friendly
20	Shadow Play	120	70695 Kennedy Place	Pre-season Friendly
21	Urban Jungle	120	39 Erie Parkway	Spring Training Game
22	Shadow Play	45	1414 Anzinger Park	Practice Scrimmage
23	Golden Hour	30	877 Jenifer Place	Charity Exhibition Match
24	Prism of Style	180	3 Fairfield Lane	Spring Training Game
25	Urban Pulse	30	5587 Charing Cross Way	Charity Exhibition Match

Figure 6.6: Exhibition's Table

Exhibition ID	Model ID
1	8
2	13
3	21
4	19
5	3
6	25
7	24
8	23
9	22
10	21
11	20
12	19
13	18
14	17
15	16
16	15
17	14
18	13
19	12
20	11
21	10
22	9
23	8
24	7
25	6
4	5
11	4
23	3
17	2
6	1

Figure 6.7: Model_Exhibition’s Table

Year
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022

Figure 6.8: Competition's Table

ID	Production_Time	Color	Fabric_Amount	Model_ID	Designer_ID
1	1	Red	5	1	1
2	4	Turquoise	5	2	2
3	3	Goldenrod	1	3	3
4	5	Turquoise	1	4	4
5	5	Green	4	5	5
6	1	Blue	4	6	6
7	1	Orange	1	7	7
8	1	Violet	1	8	8
9	3	Crimson	1	9	9
10	4	Yellow	3	10	10
11	5	Green	4	11	11
12	1	Goldenrod	4	12	12
13	4	Orange	3	13	13
14	5	Indigo	4	14	14
15	3	Purple	5	15	15
16	3	Orange	3	16	16
17	5	Khaki	1	17	17
18	4	Indigo	4	18	18
19	3	Teal	4	19	19
20	1	Turquoise	4	20	20
21	1	Pink	1	21	21
22	4	Mauv	5	22	22
23	2	Indigo	5	23	23
24	1	Orange	3	24	24
25	4	Teal	4	25	25

Figure 6.9: Dress's Table

Dress ID	Size ID
1	3
2	5
3	3
4	3
5	2
6	5
7	5
8	1
9	2
10	2
11	5
12	3
13	2
14	5
15	4
16	2
17	5
18	4
19	1
20	4
21	4
22	3
23	3
24	3
25	1

Figure 6.10: Dress_Size's Table

Dress ID	Fabric ID
1	25
2	1
3	22
4	3
5	25
6	14
7	16
8	22
9	21
10	16
11	13
12	14
13	11
14	20
15	15
16	20
17	15
18	4
19	15
20	23
21	2
22	4
23	5
24	1
25	12

Figure 6.11: Dress_Fabric's Table

Year	Dress ID	Winner
1998	12	TRUE
1999	11	FALSE
2000	7	FALSE
2001	22	TRUE
2002	10	FALSE
2003	1	FALSE
2004	25	FALSE
2005	3	TRUE
2006	8	TRUE
2007	19	FALSE
2008	9	FALSE
2009	20	FALSE
2010	2	TRUE
2011	4	FALSE
2012	14	TRUE
2013	21	FALSE
2014	6	TRUE
2015	13	FALSE
2016	24	TRUE
2017	16	FALSE
2018	18	FALSE
2019	15	TRUE
2020	5	FALSE
2021	17	TRUE
2022	23	TRUE

Figure 6.12: Competition_Dress's Table

7 | Queries

In this final chapter we will utilise MySQL Workbench platform to ask different "queries" to our database and monitor the outputs. The queries will be presented in textual form and translated into SQL statements, while the output should clearly and correctly answer the initial textual query.

These queries must follow the specific guidelines defined in the project assignment and contain specific combinations of SQL statements, but must also produce interesting results that could be applied to a real-life scenario.

Each sub-section of this chapter will be dedicated to each of the different SQL statements combinations required by the project assignment, and will present and analyze the queries we have created and their output.

7.1. WHERE

The **WHERE** statement is used to filter out the output based on a given condition.

Query 1

Select the fabrics where the price is greater than 150.

```
1 SELECT *
2 FROM Fabric
3 WHERE PriceM2 > 150
```

ID	Name	Description	PriceM2
2	Silk	Luxurious	179.28
6	Satin	Shiny	187.13
9	Chiffon	Stretchy	199.22
14	Cashmere	Cozy	181.17
18	Lycra	Vibrant	172.32
25	Corduroy	Structured	192.17

Figure 7.1: WHERE Query 1 Output

In this query, we display all output rows and columns by using the * value in the **SELECT** statement.

We start the construction of our output table **FROM** the "Fabric" table and filter for rows with PriceM2 higher than 150.

This will output all the columns in "Fabric" table, but only the rows in which PriceM2 is higher than 150.

Query 2

Select the dresses where the color is orange.

```
1 SELECT *
2 FROM Dress
3 WHERE Color = 'Orange';
```

ID	Production_Time	Color	Fabric_Amount	Model_ID	Designer_ID
7	1	Orange	1	7	7
13	4	Orange	3	13	13
16	3	Orange	3	16	16
24	1	Orange	3	24	24
26	1	Orange	4	23	1

Figure 7.2: WHERE Query 2 Output

Much like the previous query, we **SELECT** all columns **FROM** the "Dress" table, but displaying only the rows **WHERE** the Color is "Orange"

7.2. WHERE, LIMIT, LIKE

LIMIT outputs only the first n results of the resulting table.

LIKE allows filtering values based on their similarity with the chosen input.

Query 1

Select the models whose name starts with letter G, limit to the first 3 names.

```
1 SELECT Name
2 FROM Model
3 WHERE Name LIKE 'G%'
4 LIMIT 3
```


Name
Ginny
Goldia
Goldie

Figure 7.3: WHERE, LIMIT, LIKE Query 1 Output

This query displays only the "Name" column from the "Model" table. The output is filtered for Names that start with the letter G and is **LIMITED** to showcase only the first 3 results.

Query 2

Select the designers whose stage name starts with letter S, limit to 4 names.

```
1 SELECT Stage_Name
2 FROM Designer
3 WHERE Stage_Name LIKE 'S%'
4 LIMIT 4
```

Stage_Name
Star
Shadow
Stella
Sonia

Figure 7.4: WHERE, LIMIT, LIKE Query 2 Output

Similar to the last one, this query displays only the name of the first 4 designers whose name starts with the letter S.

7.3. WHERE, IN, Nested Query

IN allows you to specify multiple values in a **WHERE** clause.

A **NESTED QUERY** is computed before the main query and can be used to perform advanced filtering.

Query 1

Select all the models who have worn a dress made from cotton fabric.

```

1 SELECT Model.Name , Model.Surname
2 FROM Dress
3 JOIN Model ON Dress.Model_ID = Model.Personal_ID
4 WHERE ID IN (
5     SELECT Dress_ID
6     FROM Dress_Fabric
7     JOIN Fabric ON Dress_Fabric.Fabric_ID = Fabric.ID
8     WHERE Name = 'Cotton'
9 )

```

Name	Surname
Sibby	Essery
Orv	Delamaine

Figure 7.5: WHERE, IN, Nested Query 1 Output

From here on out queries start getting more complex.

We start from the "Dress" table and use the **JOIN** statement to extend it with the "Model" table, creating correspondences based on the Model ID.

We then filter out the results by only selecting the dress's IDs inside a specific list. This list is created using a nested query, inside which we use the **JOIN** statement to create correspondences between the Dress_Fabric table and the Fabric table. We filter out these results by only selecting the dresses whose Fabric ID indicates Cotton fabric.

Lastly, we combine all these conditions to only display the name of the models who wore a dress made out of cotton.

Query 2

Select the stage names of the designers who have designed at least one dress that has won a competition.

```

1 SELECT DISTINCT Designer.Stage_Name
2 FROM Dress
3 JOIN Designer ON Dress.Designer_ID = Designer.Personal_ID
4 WHERE ID IN (
5     SELECT Dress_ID

```

```

6      FROM Competition_Dress
7      WHERE Winner = TRUE
8  )

```

Stage_Name
Gio
Eva
Luna
Star
Shadow
Lola
Luis
Emily
Bobby
Andy
Heidi
Blaze

Figure 7.6: WHERE, IN, Nested Query 2 Output

In this query's nested query we select the list of Dress's IDs from the "Competition_Dress" table whose "Winner" attribute possesses a TRUE value.

We then use this list to filter out winner and loser dresses **FROM** the "Dress" table and **JOIN** it with "Designer" table to only display the stage name of the designers who have made at least one winner dress.

7.4. GROUP BY, 1 JOIN, AS

GROUP BY combines different rows of the table depending on the shared value in a specified column.

JOIN extends the table upon which we are working. Connections must be made between rows referencing [Primary](#) and [Foreign](#) keys.

AS creates a temporary alias for any row, column of table for the duration of the query. It can be used in the **SELECT** statement to change the output display.

Query 1

Find the number of models who participated in each exhibition.

```

1  SELECT M_Ex.Exhibition_ID, COUNT(*) AS Total_Models

```

```

2 FROM Model
3 JOIN Model_Exhibition AS M_Ex ON Model.Personal_ID = M_Ex.Model_ID
4 GROUP BY M_Ex.Exhibition_ID

```

Exhibition_ID	Total_Models
6	2
25	1
17	2
11	2
3	1
10	1
1	1
23	2
19	1
13	1
8	1
24	1
20	1
15	1
7	1
5	1
21	1
16	1
14	1
9	1
4	2
22	1
2	1
18	1
12	1

Figure 7.7: GROUP BY, 1 JOIN, AS Query 1 Output

In this query we start **FROM** the "Model" table **JOINED** with the "Model_Exhibition" table and **GROUP** the rows **BY** the Exhibition ID.

This allows us to use the aggregate function **COUNT** to return the number of total models for each exhibition in the output. To display the name "Total Models" in place of the **COUNT** function, we use the **AS** statement.

Query 2

Find the number of dresses produced by each fashion designer and display the designer's name along with the corresponding dress count.

```

1 SELECT Designer.Name, Designer.Surname, Designer.Stage_name, COUNT(ID)
   AS Total_Dresses
2 FROM Dress
3 JOIN Designer ON Designer.Personal_ID = Dress.Designer_ID
4 GROUP BY Designer.Personal_ID

```

Name	Surname	Stage_name	Total_Dresses
Terra	Akast	Gio	2
Reeva	Gullane	Eva	1
Myrlene	MacDermid	Luna	1
Blondy	Evreux	Axel	1
Pegeen	Adcock	Aldo	1
Tull	Crosland	Star	1
Rancell	Antowski	Echo	1
Ludovico	Knibley	Shadow	1
Tally	Darwent	Stella	1
Ingamar	Sabban	Agata	1
Leshia	Davidson	Amy	1
Jamil	Finnigan	Lola	1
Delila	Narramore	Sonia	1
Mort	Rubinivitz	Luis	1
Jasen	McLeoid	Emily	1
Dulcinea	Brabant	Wilson	1
Marianna	Ziemens	Bobby	1
Waylen	Macer	Sparkle	1
Malvina	Phil	Rocky	1
Jennie	Kittow	Iris	1
Cherrita	Sharpin	Chris	1
Loralie	Alloway	Andy	1
Danna	Methering...	Heidi	1
Andee	Starrs	Blaze	1
Goldina	Palffy	Wanda	1

Figure 7.8: GROUP BY, 1 JOIN, AS Query 2 Output

Similar to the previous query, we **GROUP BY** the Designer's IDs the **JOINED** tables Dress and Designer. We then return the count of dresses made by each Designer, alongside the Designer's name and stage name. To display the name "Total Dresses" in place of the **COUNT** function, we use the **AS** statement.

7.5. WHERE, GROUP BY

Query 1

Find the average height of models born after 2000 for each exhibition.

```

1 SELECT Exhibition.Title, AVG(Model.Height)
2 FROM Model_Exhibition AS M_Ex
3 JOIN Model ON M_Ex.Model_ID = Model.Personal_ID
4 JOIN Exhibition ON M_Ex.Exhibition_ID = Exhibition.ID
5 WHERE Model.Birth_Date > '2001-01-01'
6 GROUP BY Exhibition.Title

```

Title	AVG(Model.Height)
Shadow Play	167.680000
Urban Pulse	169.270000
Golden Hour	181.450000
Prism of Style	178.330000
Bright Allure	166.090000

Figure 7.9: WHERE, GROUP BY Query 1 Output

GROUPING BY Exhibition title the **JOINED** tables "Model_Exhibition", "Model" and "Exhibition" based on corresponding IDs, we filter out the results by only selecting models born after the year 2000 and return the the average height of these models for each exhibition they attended.

Query 2

Count the number of models taller than 170 cm, grouped by the size they are wearing.

```

1 SELECT Size.Description, COUNT(*) AS Number_of_Models
2 FROM Model
3 RIGHT JOIN Size ON Model.Size_ID = Size.ID
4 WHERE Model.Height > 170
5 GROUP BY Size.ID

```

Description	Number_of_Models
X-Small	3
Medium	3
Large	4
Small	3
X-Large	3

Figure 7.10: WHERE, GROUP BY Query 2 Output

Following the previous query, we **GROUP BY** size the **JOINED** tables "Model" and "Size". We then filter out for the models higher than 170 cm and return the description of each size and the **COUNT** of models for each grouping.

7.6. GROUP BY, HAVING, AS

HAVING functions the same as the **WHERE** statement, but it can be used with aggregate functions and is applied after a **GROUP BY** statement.

Query 1

Find how many models wear each size and return only the sizes worn by at least two models.

```
1 SELECT Size.Description, COUNT(*) AS Total_Models
2 FROM Size
3 JOIN Model ON Size.ID = Model.Size_ID
4 GROUP BY Size.ID
5 HAVING Total_Models >= 2
```

Description	Total_Models
X-Small	5
Small	5
Medium	5
Large	5
X-Large	5

Figure 7.11: GROUP BY, HAVING, AS Query 1 Output

Once again, we **GROUP BY** size id the **JOINED** tables "Dress_Size", "Size" and "Model". We **COUNT** the results for each grouping and only display the counts higher or equal to 2.

Query 2

Find the exhibitions in which more than one model participated.

```

1 SELECT Ex.ID, Ex.Title, COUNT(*) AS Total_Model
2 FROM Model_Exhibition AS M_Ex
3 JOIN Exhibition AS Ex ON M_Ex.Exhibition_ID = Ex.ID
4 GROUP BY Ex.ID
5 HAVING Total_Model >= 2

```

ID	Title	Total_Model
4	Midnight Glamour	2
6	Naive Chic	2
11	Rebel Chic	2
17	Echoes of Couture	2
23	Golden Hour	2

Figure 7.12: GROUP BY, HAVING, AS Query 2 Output

Much like the previous query, we **GROUP BY** exhibition ID the **JOINED** tables "Model_Exhibition" and "Exhibition". We then return the ID, name and total model **COUNT** of each exhibition, but only for those whose count is higher than 1.

7.7. WHERE, GROUP BY, HAVING, AS

Query 1

Find the fabrics that are made from at least two other fabrics.

```

1 SELECT F1.Name AS Fabric1_Name, COUNT(F2.ID) AS Connected_Fabrics
2 FROM Fabric AS F1
3 JOIN Fabric_Fabric ON F1.ID = Fabric_Fabric.Fabric_ID1
4 JOIN Fabric AS F2 ON Fabric_Fabric.Fabric_ID2 = F2.ID
5 GROUP BY F1.ID
6 HAVING COUNT(F2.ID) >= 2;

```

We **JOIN** the Fabric table with itself and the Fabric_Fabric table, using different aliases for each table to avoid conflicting IDs. We **GROUP BY** each F1 ID, the left ID in the Fabric_Fabric tables, and **COUNT** the number of associated F2 IDs, the right Fabric_Fabric IDs.

We return the Name of each F1 Fabric and the **COUNT** of F2 fabrics **AS** "Connected Fabrics", but only if this count is higher or equal than 2.

Fabric1_Name	Connected_Fabrics
Cashmere	3
Denim	2
Chiffon	2
Jersey	3
Corduroy	2
Satin	2
Gabardine	2
Lycra	2

Figure 7.13: WHERE, GROUP BY, HAVING, AS Query 1 Output

Query 2

Find the average PriceM2 and the fabrics with an average price greater than 50, used to make the winning dresses in the competitions.

```

1 SELECT Fabric.Name AS Fabric_Name, AVG(Fabric.PriceM2) AS
   Avg_Fabric_Price
2 FROM Dress
3 JOIN Competition_Dress ON Dress.ID = Competition_Dress.Dress_ID
4 JOIN Dress_Fabric ON Dress.ID = Dress_Fabric.Dress_ID
5 JOIN Fabric ON Dress_Fabric.Fabric_ID = Fabric.ID
6 WHERE Competition_Dress.Winner = TRUE
7 GROUP BY Fabric.Name
8 HAVING AVG(Fabric.PriceM2) > 50

```

Fabric_Name	Avg_Fabric_Price
Corduroy	192.170000
Cotton	62.010000
Suede	50.360000
Cashmere	181.170000
Wool	98.120000

Figure 7.14: WHERE, GROUP BY, HAVING, AS Query 2 Output

After **JOINING** together the "Dress", "Competition_Dress", "Dress_Fabric" and "Fabric" tables, we select only the dresses that have won a competition.

We then **GROUP** these results **BY** Fabric name and return the name of the fabric **AS** "Fabric Name" and the average price of these Fabrics, but only if the average is higher than 50.

7.8. WHERE, Nested Query, GROUP BY

Query 1

Find, for each size, whether a dress won a competition and, if so, how many dresses per size.

```

1 SELECT Size.Description AS 'Size', COUNT(*) AS TotalWinnerDresses
2 FROM Dress
3 JOIN Dress_Size ON Dress_Size.Dress_ID = Dress.ID
4 JOIN Size ON Size.ID = Dress_Size.Size_ID

```

```

5 WHERE Dress.ID IN (
6     SELECT Dress_ID
7     FROM Competition_Dress
8     WHERE Winner = TRUE
9 )
10 GROUP BY Size.Description

```

We start from the **JOINED** tables "Dress", "Dress_Size" and "Size" and filter the result based on a nested query list.

The nested query returns the list of Dress IDs that correspond to a competition winner dress.

We group these results by Size description and return the **COUNT** of total winner dresses for each size.

Size	TotalWinnerDresses
Medium	6
X-Large	4
X-Small	1
Large	1

Figure 7.15: WHERE, Nested Query, GROUP BY Query 1 Output

Query 2

Find, for each size, the number of models who participated in at least one exhibition and were born in or after the year 2000.

```

1 SELECT Size.Description AS 'Size', COUNT(*) AS
   TotalExhibitionParticipants
2 FROM Model
3 JOIN Size ON Size.ID = Model.Size_ID
4 WHERE Birth_Date > '2000-01-01' AND Personal_ID IN (
5     SELECT Model_ID
6     FROM Model_Exhibition
7 )
8 GROUP BY Size.Description

```

After **JOINING** the "Model" and "Size" tables, we filter out the results using a compounded **WHERE** statement.

We select all models that have been born in or after the year 2000 **AND** are present in a list defined by a nested query. The nested query returns the list of Models that have attended at least an exhibition.

We then group the rows **BY** size description and return the size description **AS** "Size" and the **COUNT** of attendees **AS** "Total Exhibition Participants".

Size	TotalExhibitionParticipants
X-Small	2
Small	2
X-Large	1

Figure 7.16: WHERE, Nested Query, GROUP BY Query 2 Output

7.9. WHERE, GROUP BY, HAVING, 1 JOIN

Query 1

Find how many exhibitions of the same style (based on description) took place, and for each style, how many unique models participated. Additionally, consider only exhibitions that lasted less than 2 hours and had at least three models participating.

```

1 SELECT COUNT(Title) AS Total_Exhibitions, Ex.Description, COUNT(DISTINCT
   M_Ex.Model_ID) AS Total_Distinct_Models
2 FROM Exhibition AS Ex
3 JOIN Model_Exhibition AS M_Ex ON M_Ex.Exhibition_ID = Ex.ID
4 WHERE Ex.Duration < 120
5 GROUP BY Ex.Description
6 HAVING Total_Distinct_Models > 2

```

Total_Exhibitions	Description	Total_Distinct_Models
6	Practice Scrimmage	4
4	Pre-season Friendly	4
5	Spring Training Game	5

Figure 7.17: WHERE, GROUP BY, HAVING, 1 JOIN Query 1 Output

Much like previous queries, we **JOIN** the "Exhibition" and "Model_Exhibition" tables and filter for exhibitions having a duration of less than 120 minutes.

We then group by for the description of each exhibition and return the **COUNT** of exhibitions and the **COUNT** of every **DISTINCT** model for each type of exhibition.

Query 2

Calculate the average production time of dresses for each designer, considering only blue and green dresses. Then, filter the results to show only designers whose average production time exceeds 2 days.

```

1 SELECT Designer.Personal_ID, Designer.Stage_Name, AVG(Dress.
   Production_Time) AS Average_Production_Time
2 FROM Dress
3 JOIN Designer ON Dress.Designer_ID = Designer.Personal_ID
4 WHERE Dress.Color = 'Blue' OR Dress.Color = 'Green'
5 GROUP BY Designer.Personal_ID
6 HAVING Average_Production_Time > 2

```

Personal_ID	Stage_Name	Average_Production_Time
5	Aldo	5.0000
11	Amy	5.0000

Figure 7.18: WHERE, GROUP BY, HAVING, 1 JOIN Query 2 Output

After **JOINING** the "Dress" and "Designer" tables, we filter for dresses colored "Blue" **OR** "Green" and group by the Designer IDs. We then return the name of each designer alongside the average production time of their dresses, but only if this average is higher than 2.

7.10. WHERE, GROUP BY, HAVING, 2 JOINS

Query 1

Find the designers whose dresses won at least one competition and how many winning dresses they achieved.

```

1 SELECT Designer.Personal_ID, Designer.Stage_Name, COUNT(*) AS
   Total_Winner_Dresses
2 FROM Dress
3 JOIN Designer ON Designer.Personal_ID = Dress.Designer_ID
4 JOIN Competition_Dress ON Competition_Dress.Dress_ID = Dress.ID
5 WHERE winner = TRUE
6 GROUP BY Designer.Personal_ID
7 HAVING Total_Winner_Dresses >= 1

```

We join the Dress table with the "Competition_Dress" and "Designer" tables based on correspondences with Designer_IDs and Dress_IDs. We apply the condition "Winner =

Personal_ID	Stage_Name	Total_Winner_Dresses
1	Gio	2
2	Eva	1
3	Luna	1
6	Star	1
8	Shadow	1
12	Lola	1
14	Luis	1
15	Emily	1
17	Bobby	1
22	Andy	1
23	Heidi	1
24	Blaze	1

Figure 7.19: WHERE, GROUP BY, HAVING, 2 JOINS Query 1 Output

TRUE ", removing all non-winner rows from our table.

Afterwards, we group by Designer_ID and display the designers with more than one winner dress, alongside their total number of winner dresses.

Query 2

Find the models who participated in more than one exhibition, don't consider the appearances in the exhibitions with dresses colored in goldenrod.

```

1 SELECT Model.Personal_ID, Model.Name, Model.Surname, COUNT(*) AS
   Total_Exhibitions
2 FROM Dress
3 JOIN Model ON Model.Personal_ID = Dress.Model_ID
4 JOIN Model_Exhibition AS exhib ON exhib.Model_ID = Model.Personal_ID
5 WHERE NOT Color = 'Goldenrod'
6 GROUP BY Dress.Model_ID
7 HAVING Total_Exhibitions > 1

```

We remove all dresses "Goldenrod" colored from the **JOINED** tables "Dress", "Model" and "Model_Exhibition". We then **GROUP BY** the Model ID and return the name of each model alongside the **COUNT** of its attended exhibitions, but only if that count is higher than 1.

Personal_ID	Name	Surname	Total_Exhibitions
1	Erick	Tapp	2
2	Sibby	Essery	3
6	Terry	Cohrs	2
11	Ignacius	Goatman	3
18	Adorne	Couche	2
23	Goldie	Hourican	2
24	Orv	Delamaine	2

Figure 7.20: WHERE, GROUP BY, HAVING, 2 JOINS Query 2 Output

Query 3

Find the total number of dresses created by each designer that used the following fabrics: tulle, silk and linen. Only consider designers whose average production time of these dresses is higher than 3 days.

```

1 SELECT Designer.Stage_Name , COUNT(*) AS Total_Dresses
2 FROM Dress
3 JOIN Designer ON Dress.Designer_ID = Designer.Personal_ID
4 WHERE Dress.ID IN (
5     SELECT DISTINCT(Dress_Fabric.Dress_ID)
6     FROM Dress_Fabric
7     JOIN Fabric ON
8     Dress_Fabric.Fabric_ID = Fabric.ID
9     WHERE Fabric.Name IN ('Linen', 'Tulle', 'Silk')
10 )
11 GROUP BY Designer.Personal_ID
12 HAVING AVG(production_time) > 3

```

Stage_Name	Total_Dresses
Axel	1
Wanda	1

Figure 7.21: WHERE, GROUP BY, HAVING, 2 JOINS Query 3 Output

In this query, we start by selecting the Dresses present in the list defined by the nested query. This nested query returns the list of dresses made with "Linen", "Tulle" and "Silk" fabric.

We then join this output with the "Desiger" table and group by the Designer IDs. We

return the name of the designer and the **COUNT** of total dresses, but only if the average of the production time of the qualifying dresses is higher than 3 days.

List of Figures

2.1	ER Model	3
4.1	Relational Model by MySQL Workbench	21
6.1	Size's Table	31
6.2	Designer's Table	32
6.3	Fabric's Table	33
6.4	Model's Table	34
6.5	Fabric_Fabric's Table	35
6.6	Exhibition's Table	36
6.7	Model_Exhibition's Table	37
6.8	Competition's Table	38
6.9	Dress's Table	39
6.10	Dress_Size's Table	40
6.11	Dress_Fabric's Table	41
6.12	Competition_Dress's Table	42
7.1	WHERE Query 1 Output	43
7.2	WHERE Query 2 Output	44
7.3	WHERE, LIMIT, LIKE Query 1 Output	45
7.4	WHERE, LIMIT, LIKE Query 2 Output	45
7.5	WHERE, IN, Nested Query 1 Output	46
7.6	WHERE, IN, Nested Query 2 Output	47
7.7	GROUP BY, 1 JOIN, AS Query 1 Output	48
7.8	GROUP BY, 1 JOIN, AS Query 2 Output	49
7.9	WHERE, GROUP BY Query 1 Output	50
7.10	WHERE, GROUP BY Query 2 Output	51
7.11	GROUP BY, HAVING, AS Query 1 Output	52
7.12	GROUP BY, HAVING, AS Query 2 Output	52
7.13	WHERE, GROUP BY, HAVING, AS Query 1 Output	53
7.14	WHERE, GROUP BY, HAVING, AS Query 2 Output	54

7.15	WHERE, Nested Query, GROUP BY Query 1 Output	55
7.16	WHERE, Nested Query, GROUP BY Query 2 Output	56
7.17	WHERE, GROUP BY, HAVING, 1 JOIN Query 1 Output	56
7.18	WHERE, GROUP BY, HAVING, 1 JOIN Query 2 Output	57
7.19	WHERE, GROUP BY, HAVING, 2 JOINS Query 1 Output	58
7.20	WHERE, GROUP BY, HAVING, 2 JOINS Query 2 Output	59
7.21	WHERE, GROUP BY, HAVING, 2 JOINS Query 3 Output	59

List of Tables

4.1	Part 1: Table "Model"	14
4.2	Part 2: Table "Model"	14
4.3	Part 1: Table "Designer"	15
4.4	Part 2: Table "Designer"	15
4.5	Tabel "Fabric"	16
4.6	Table "Fabric_Fabric"	16
4.7	Table "Exhibition"	17
4.8	Table "Model_Exhibition"	17
4.9	Tabel "Size"	17
4.10	Table "Competition"	18
4.11	Table "Competition_Dress"	18
4.12	Table "Dress"	19
4.13	Table "Dress_Size"	20
4.14	Table "Dress_Fabric"	20

